

CS4234: Optimization Algorithms

@Wxy2003-xy

Contents

0.1	Prerequisite map	2
0.2	The optimization viewpoint	2
0.3	Asymptotics, input size, and cost models	2
0.4	Proof obligations for algorithms	3
0.5	Algorithm-design tools retained from CS3230	3
0.6	Probability toolkit for randomized optimization	4
0.7	Logic and constraint modelling from CS4269	4
0.8	From Boolean constraints to binary optimization	5
0.9	Complexity classes and reductions	5
0.10	A checklist for the rest of CS4234	5
1	Lecture 1: Approximation Algorithm	6
1.1	Definiton	6
1.2	General Approach	6
1.3	Example: Greedy algorithms on 0/1 knapsack	6
1.4	Strategy: Pick the best among candidate algorithm results	8
1.5	Example: Subset-sum	9
1.6	Heuristic	10
1.7	Set cover	10
1.8	Vertex Cover as a special case of Set cover	12
1.9	Fully polynomial-time approximation scheme	14
1.9.1	Woeginger FPTAS	14
1.10	Randomized approximation algorithm	14
1.11	Example: (Minimum) Vertex Cover	14
1.11.1	Greedy approach	14
1.12	Example: MAX-SAT	15
1.13	Inapproximability	15

Introduction and Prerequisites

0.1 Prerequisite map

Source	Reusable idea	Role in CS4234
CS3230	Asymptotic analysis, recurrences, and input encoding	State running times in the true input length and distinguish polynomial from pseudo-polynomial algorithms.
CS3230	Loop invariants, induction, exchange arguments, and optimal substructure	Prove feasibility, optimality, approximation ratios, and termination.
CS3230	Greedy algorithms, dynamic programming, graph algorithms, and randomization	Supply exact subroutines and recurring algorithm-design patterns.
CS3230/CS4269	Polynomial-time reductions and NP-completeness	Explain why an exact polynomial-time algorithm may be unlikely and motivate relaxations, approximation, or restricted cases.
CS4269	Propositional semantics, satisfiability, and formula encodings	Turn combinatorial choices into Boolean or 0–1 variables with explicit constraints.

Topics such as compactness, Hilbert systems, first-order logic, and computability remain useful background, but they are not part of this compact optimization prerequisite.

0.2 The optimization viewpoint

An optimization problem separates the description of legal solutions from the criterion used to rank them.

Definition 0.1 (Optimization problem). For an instance I , let $F(I)$ be its feasible set and let $c_I: F(I) \rightarrow \mathbb{R}$ be its objective. A minimization problem asks for

$$\text{OPT}(I) = \min_{x \in F(I)} c_I(x),$$

while a maximization problem replaces min by max. A complete answer may include both an optimal value and a witness x^* attaining that value.

The same problem often appears in three forms:

- **decision:** is there a feasible solution of value at most B (minimization), or at least B (maximization)?
- **search:** find any feasible solution, possibly satisfying a threshold;
- **optimization:** find the best feasible solution.

For an integer-valued objective known to lie in $[L, U]$, an optimization value can often be recovered using $O(\log(U - L + 1))$ calls to the decision problem. This is a polynomial-time reduction when the endpoints have polynomial bit length; the bound and encoding must be stated rather than silently assumed.

A standard model. Many problems can be written as

$$\min\{c^\top x : x \in F\},$$

where x records choices, F records constraints, and c records costs. The domain of x matters: $x \in \{0, 1\}^n$ is discrete, whereas $x \in [0, 1]^n$ is a continuous relaxation. Enlarging F can make a problem easier to solve, but may produce a solution that is not feasible for the original problem.

0.3 Asymptotics, input size, and cost models

For eventually non-negative functions,

$$\begin{aligned} f \in O(g) &\iff \exists C > 0, \exists n_0, \forall n \geq n_0 : f(n) \leq Cg(n), \\ f \in \Omega(g) &\iff \exists c > 0, \exists n_0, \forall n \geq n_0 : f(n) \geq cg(n), \\ f \in \Theta(g) &\iff f \in O(g) \cap \Omega(g). \end{aligned}$$

Useful limit tests are

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f \in o(g), \quad 0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty \Rightarrow f \in \Theta(g).$$

The robust versions use $\limsup$ and $\liminf$ when the ratio oscillates.

Bit complexity. Running time is measured against the length of the encoded input, not merely the numerical value of its parameters. An integer W requires $\Theta(\log(W + 1))$ bits in binary. Consequently, a running time such as $O(nW)$ is polynomial in the *value* W but can be exponential in its bit length. Such an algorithm is **pseudo-polynomial**. This distinction is central for knapsack-style dynamic programs and many weakly NP-hard optimization problems.

Cost accounting. For every claimed bound, identify:

1. the input-size parameters and their encodings;
2. the number of iterations, states, or recursive calls;
3. the cost of each operation or oracle call; and
4. any hidden dependence on numerical precision.

A recurrence such as $T(n) = aT(n/b) + f(n)$ is appropriate for recursive decomposition. Iterative optimization algorithms are often clearer as

$$\text{preprocessing cost} + (\text{number of iterations})(\text{cost per iteration}).$$

0.4 Proof obligations for algorithms

The loop-invariant template from CS3230 remains the default proof pattern: *initialization*, *maintenance*, and *termination*. For a recursive algorithm, replace the invariant by induction on the instance size. For optimization, separate the claims that are easy to accidentally conflate.

1. **Feasibility:** the returned solution lies in $F(I)$.
2. **Value guarantee:** its objective satisfies the promised bound.
3. **Optimality or approximation:** compare it with every feasible solution, a lower/upper bound, or an optimal witness.
4. **Termination and complexity:** the procedure finishes within the claimed resources under the stated computational model.

For a non-negative objective and an α -approximation with $\alpha \geq 1$, the usual conventions are

$$\begin{aligned} \text{minimization: } & \text{OPT}(I) \leq \text{ALG}(I) \leq \alpha \text{OPT}(I), \\ \text{maximization: } & \text{OPT}(I)/\alpha \leq \text{ALG}(I) \leq \text{OPT}(I). \end{aligned}$$

Feasibility gives one side of each inequality; the real proof is the other side. A common strategy is to compare the algorithm against a certificate bound $L(I) \leq \text{OPT}(I)$ for minimization or $U(I) \geq \text{OPT}(I)$ for maximization.

Reusable proof patterns.

- **Exchange argument:** transform an optimum so that it agrees with the greedy choice without worsening its value.
- **Optimal substructure:** after fixing a decision, the remainder is an optimal solution to a well-defined subproblem.
- **Charging argument:** charge the cost of the computed solution to parts of an optimum, with bounded charge per part.
- **Relaxation bound:** compare an integral solution with the value of a relaxation whose feasible region contains all integral solutions.

0.5 Algorithm-design tools retained from CS3230

Dynamic programming. A DP proof should state the subproblem, recurrence, base cases, evaluation order, and recovery of a witness. If $D[s]$ denotes the best value for state s , the key equality is justified by partitioning all feasible solutions by their last decision:

$$D[s] = \text{opt}_{a \in A(s)} \{ \text{cost}(s, a) + D[\text{next}(s, a)] \}.$$

The running time is the number of reachable states times the transition cost. State coordinates that range up to a numeric value such as a capacity W are where pseudo-polynomial behaviour usually enters.

Greedy algorithms. A locally best choice is not a proof. Correctness normally requires an exchange argument, a stays-ahead invariant, or a structural property such as a cut property. When no such proof exists, the same greedy rule may still be useful as a heuristic or approximation algorithm, but its guarantee must be analysed separately.

Graph subroutines. Shortest paths, minimum spanning trees, maximum flow, matching, and connectivity recur as exact subroutines and as sources of relaxations. Always record their preconditions: for example, Dijkstra’s algorithm requires non-negative edge weights, whereas Bellman–Ford permits negative edges but must detect reachable negative cycles.

0.6 Probability toolkit for randomized optimization

Let $X_1, \dots, X_n$ be random variables and let $X = \sum_i X_i$. The most reusable facts from CS3230 are

$$\begin{aligned} \mathbb{E}[X] &= \sum_i \mathbb{E}[X_i] && \text{(no independence required),} \\ \Pr\left[\bigcup_i A_i\right] &\leq \sum_i \Pr[A_i] && \text{(union bound),} \\ \Pr[X \geq t] &\leq \frac{\mathbb{E}[X]}{t} && \text{for } X \geq 0 \text{ (Markov).} \end{aligned}$$

Indicator variables $X_i = \mathbf{1}[A_i]$ turn counting questions into expectations because $\mathbb{E}[X_i] = \Pr[A_i]$.

For independent Bernoulli variables with $\mu = \mathbb{E}[X]$ and $0 < \delta < 1$, useful Chernoff forms are

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{-\delta^2 \mu / 2}, \quad \Pr[X \geq (1 + \delta)\mu] \leq e^{-\delta^2 \mu / (2 + \delta)}.$$

These concentration inequalities require stronger assumptions than linearity of expectation; independence must be checked rather than silently assumed.

If one independent run fails with probability at most $q < 1$, then t runs all fail with probability at most q^t . This is the amplification principle seen in Freivalds’ algorithm. In optimization it also supports randomized rounding: solve a fractional relaxation, interpret fractional coordinates as selection probabilities, and prove objective and feasibility bounds using expectation plus concentration.

0.7 Logic and constraint modelling from CS4269

A propositional formula has *syntax*; a valuation gives it *semantics*. For a valuation v ,

- $v \models \phi$ means that v satisfies ϕ ;
- ϕ is satisfiable if some v satisfies it;
- ϕ is valid if every v satisfies it;
- $\Gamma \models \phi$ means every model of Γ is a model of ϕ ; and
- $\phi \equiv \psi$ means the two formulas have the same models.

The distinction between satisfiability and validity matters: a feasible solution corresponds to *one* satisfying assignment, not to a formula that is true under all assignments.

Constraint-encoding recipe. The graph-colouring, Hamiltonian-path, clique, exact-cover, and scheduling encodings in CS4269 share one pattern:

1. introduce a variable for each atomic choice;
2. write separate constraints for existence, uniqueness, compatibility, and global requirements;
3. conjoin the constraints into one formula;
4. prove a two-way correspondence between models and feasible solutions;
5. prove that the encoding has polynomial size and is computable in polynomial time.

For variables $p_1, \dots, p_k$, “exactly one” is

$$\left(\bigvee_{i=1}^k p_i\right) \wedge \left(\bigwedge_{1 \leq i < j \leq k} (\neg p_i \vee \neg p_j)\right).$$

The first part gives existence; the pairwise clauses give uniqueness.

0.8 From Boolean constraints to binary optimization

Set $x_i = 1$ when proposition p_i is true and $x_i = 0$ otherwise. Common logical constraints then become linear inequalities:

Boolean constraint	0-1 linear form
$p_1 \vee \cdots \vee p_k$	$x_1 + \cdots + x_k \geq 1$
$p \rightarrow q$	$x_p \leq x_q$
exactly one of $p_1, \dots, p_k$	$x_1 + \cdots + x_k = 1$
$z \leftrightarrow (p \wedge q)$	$z \leq x_p, z \leq x_q, \text{ and } z \geq x_p + x_q - 1$
clause $\ell_1 \vee \cdots \vee \ell_k$	$\sum_{\ell_i=p_j} x_j + \sum_{\ell_i=\neg p_j} (1 - x_j) \geq 1$

Example: minimum vertex cover. For a graph $G = (V, E)$, let x_v indicate whether vertex v is selected:

$$\min \sum_{v \in V} x_v \quad \text{subject to} \quad x_u + x_v \geq 1 \quad \forall \{u, v\} \in E, \quad x_v \in \{0, 1\}.$$

Each edge constraint is the clause $p_u \vee p_v$. Replacing $x_v \in \{0, 1\}$ by $0 \leq x_v \leq 1$ gives a linear relaxation. Its feasible region is larger, so for this minimization problem its optimum is a lower bound on the integral optimum.

0.9 Complexity classes and reductions

Definition 0.2 (Core classes). P is the class of decision problems solvable in polynomial time. NP is the class whose YES instances have polynomial-size certificates verifiable in polynomial time. A problem B is NP-hard if every problem in NP polynomial-time reduces to B ; it is NP-complete if, in addition, $B \in NP$.

A polynomial-time many-one reduction $A \leq_p B$ is a polynomial-time computable map f such that

$$x \in A \iff f(x) \in B.$$

The direction has an operational meaning: an algorithm for B would yield an algorithm for A . To prove that B is hard, reduce a known hard problem to B , not the other way around.

Every reduction proof has three obligations:

1. **construction:** define $f(x)$ completely;
 2. **correctness:** prove both directions of $x \in A \iff f(x) \in B$; and
 3. **complexity:** prove that f runs in polynomial time and its output has polynomial encoded size.
- The SAT encodings from CS4269 are reductions when they satisfy exactly these obligations. Cook–Levin supplies the central starting point: SAT is NP-complete.

Decision versus optimization hardness. NP-completeness formally applies to decision problems. To discuss an optimization problem, define its threshold version. For example, the decision version of minimum vertex cover asks whether a cover of size at most k exists. If an exact optimizer ran in polynomial time, evaluating whether its returned value is at most k would solve this decision problem in polynomial time. Thus NP-hardness of the threshold problem implies NP-hardness of exact optimization.

0.10 A checklist for the rest of CS4234

For each new problem or algorithm, record the following before doing algebra:

1. What is the instance, and what is its encoded length?
2. What are the variables, feasible solutions, and objective direction?
3. Is the task decision, search, or optimization?
4. What invariant, exchange, induction, charging, or relaxation argument proves the guarantee?
5. Is the guarantee exact, approximate, expected, or with high probability?
6. Does the runtime depend polynomially on the bit length, or only on a numerical value?
7. For a reduction, is the direction correct, is the equivalence proved both ways, and is the construction polynomial in encoded size?

1 Lecture 1: Approximation Algorithm

1.1 Definiton

p -Approximation:

- Minimization:

$$ALG \leq p \cdot OPT \implies \frac{ALG}{OPT} \leq p$$

- Maximization:

$$\frac{OPT}{p} \leq ALG \implies \frac{OPT}{ALG} \leq p$$

1.2 General Approach

1. Relax
2. Decompose
3. Compare
4. Conclude

1.3 Example: Greedy algorithms on 0/1 knapsack

We can relax 0/1 knapsack to fractional knapsack, for which prioritizing items by highest value density is optimal. Applying the same ordering to whole items is a natural first greedy approach, but it can be arbitrarily bad for 0/1 knapsack.

Value-density-first greedy

Algorithm 1: Value-density-first greedy for 0/1 knapsack

Input: Values $v_1, \dots, v_n$, weights $w_1, \dots, w_n$, capacity W

Output: A feasible item set S_D and its total value V_D

$F \leftarrow \{i \in \{1, \dots, n\} : w_i \leq W\}$;

Sort the items in F by non-increasing density v_i/w_i ;

$S_D \leftarrow \emptyset, W_D \leftarrow 0, V_D \leftarrow 0$;

foreach $i \in F$ *in density order* **do**

if $W_D + w_i > W$ **then**

break;

$S_D \leftarrow S_D \cup \{i\}$;

$W_D \leftarrow W_D + w_i$;

$V_D \leftarrow V_D + v_i$;

return (S_D, V_D) ;

Counter example

Consider

$$v = \{2, W\}$$

$$w = \{1, W\}$$

The value-density-first algorithm takes only the first item and returns value 2, while the optimal solution takes the second item and returns W .

$$\frac{OPT}{ALG} = \frac{W}{2}$$

There is no constant approximation factor, meaning this algorithm can give us arbitrarily bad result.

Identify the gap

From the counter example, we can see that a gap is caused by high-value low density (consequently high weight) items. This suggests another greedy approach: prioritize items by value rather than by density.

Algorithm 2: Value-first greedy for 0/1 knapsack

Input: Values $v_1, \dots, v_n$, weights $w_1, \dots, w_n$, capacity W

Output: A feasible item set S_V and its total value V_V

$F \leftarrow \{i \in \{1, \dots, n\} : w_i \leq W\};$

Sort the items in F by non-increasing value v_i ;

$S_V \leftarrow \emptyset, W_V \leftarrow 0, V_V \leftarrow 0;$

foreach $i \in F$ *in value order* **do**

if $W_V + w_i \leq W$ **then**

$S_V \leftarrow S_V \cup \{i\};$

$W_V \leftarrow W_V + w_i;$

$V_V \leftarrow V_V + v_i;$

return $(S_V, V_V);$

This solves the problem of overlooking a high-value item, but it can still be arbitrarily bad.

Counter example for value-first greedy

Let the capacity be $W > 2$, with one heavy item and W unit items:

$$v = \{2, \underbrace{1, 1, \dots, 1}_{W \text{ items}}\},$$
$$w = \{W, \underbrace{1, 1, \dots, 1}_{W \text{ items}}\}.$$

Value-first greedy takes the heavy item, fills the knapsack, and returns value 2. The optimal solution takes all W unit items and has value W . Therefore,

$$\frac{OPT}{ALG} = \frac{W}{2} \longrightarrow \infty.$$

Thus neither density-first nor value-first greedy has a constant approximation factor by itself.

Modified greedy: keep the better candidate

The fractional solution is a whole-item density-greedy prefix plus a fraction of the first item that does not fit. This identifies the two candidates needed for the repair: the density-greedy prefix and the best feasible singleton.

Algorithm 3: Modified two-candidate greedy for 0/1 knapsack

Input: Values $v_1, \dots, v_n$, weights $w_1, \dots, w_n$, capacity W

Output: A feasible item set S and its total value

$F \leftarrow \{i \in \{1, \dots, n\} : w_i \leq W\}$;

if $F = \emptyset$ **then**

return $(\emptyset, 0)$;

Sort the items in F by non-increasing density v_i/w_i ;

$S_G \leftarrow \emptyset, W_G \leftarrow 0, V_G \leftarrow 0$;

foreach $i \in F$ **in density order do**

if $W_G + w_i > W$ **then**

break;

$S_G \leftarrow S_G \cup \{i\}$;

$W_G \leftarrow W_G + w_i$;

$V_G \leftarrow V_G + v_i$;

$i_{\max} \leftarrow \arg \max_{i \in F} v_i$;

$S_M \leftarrow \{i_{\max}\}, V_M \leftarrow v_{i_{\max}}$;

if $V_G \geq V_M$ **then**

return (S_G, V_G) ;

else

return (S_M, V_M) ;

The value-first greedy solution always contains the best feasible singleton, so it could replace the singleton candidate and would preserve the guarantee. The singleton is used because it is the minimal repair required by the proof.

1.4 Strategy: Pick the best among candidate algorithm results

Suppose our goal is to maximize $g(x)$, where

$$g(x) = f_1(x) + f_2(x) + \dots + f_n(x)$$

But we only have an optimal solution x_i for each individual function f_i . Consider

$$g'(x) = \max_{j \in [1, n]} \left\{ \sum_{i=1}^j f_i(x_j) \right\}$$

We can show that this will give us a solution at worst n times less the optimal solution.

Proof. Suppose there exists an optimal solution x^* , such that $\forall x, g(x^*) \geq g(x)$ thus we have

$$OPT = \sum_{i=1}^n f_i(x^*)$$

Given that we have the local optimal x_i for each f_i , we also have

$$\forall i \leq n, f_i(x^*) \leq f_i(x_i)$$

$$\begin{aligned} OPT &= \sum_{i=1}^n f_i(x^*) \leq \sum_{i=1}^n f_i(x_i) \leq \sum_{j=1}^n \sum_{i=1}^n f_i(x_j) \leq n \times \max_{j \in [1, n]} \left\{ \sum_{i=1}^j f_i(x_j) \right\} = n \cdot ALG \\ ALG &\geq \frac{OPT}{n} \end{aligned}$$

□

The important constraint of picking candidate algorithms is to make sure that

$$OPT \leq \sum_i A_i$$

In the 0/1 knapsack problem, this guarantee is enforced, as the optimal solution for fractional knapsack

$$G + \alpha v_j$$

where $0 \leq \alpha \leq 1$, and v_j is the partially taken item in the greedy solution to the fractional problem. From this we have a series of inequalities:

$$G + \alpha v_j \leq G + v_j \leq G + v_{\max}$$

And as the fractional version is a relaxed version of the integral version, meaning

$$OPT \leq G + \alpha v_j$$

Our modified algorithm gives us

$$ALG = \max(G, v_{\max}) \geq \frac{G + v_{\max}}{2}$$

Overall

$$OPT \leq 2 \cdot ALG$$

1.5 Example: Subset-sum

Consider the optimization version of the subset sum problem, where given a set S , find the max sum of $S' \subseteq S$ such that $\sum S' \leq B$.

Greedy approach

An obvious greedy solution would be to select the largest numbers first.

```

Input: Set  $S$ , threshold  $B$ 
Output: Subset sum  $S'$ 
 $S' \leftarrow \emptyset$ ;
foreach  $s_i \in S$  in descending order do
    if  $B - \sum S' < s_i$  then
        return  $S'$ ;
    else
         $S' \leftarrow S' \cup \{s_i\}$ ;
         $B \leftarrow B + s_i$ ;
return  $S'$ ;

```

We can construct a counter example similarly to the highest-value first greedy approach for knapsack: Consider

$$S = \{100, 51, 51\}, B = 102$$

The greedy algorithm gives us 100, while optimal solution is 102. However in this case the greedy algorithm is already a 2-approximation.

Proof. Let the greedy sum be G , and the largest feasible number is M . Given that we select from the largest, we have $G \geq M$ OR $G = 0$. in the case of $G = 0$, meaning there is no feasible element, thus $OPT = ALG = 0$. Remaining 2 non trivial cases:

case 1: $G < \frac{B}{2}$

We will show that $G < \frac{B}{2} \implies G = OPT$. According to our algorithm, suppose a rejection happens after selection some elements, a partial sum $s \leq G$, due to over capacity, meaning

$$s + a_i > B, \quad s > B - a_i$$

Under our assumption,

$$B - a_i < s \leq G < \frac{B}{2}$$

$$a_i > \frac{B}{2}$$

Suppose the first largest item exist in thr selection,

$$G \geq a_1 \geq a_i > \frac{B}{2}$$

contradition, as we assume $G < \frac{B}{2}$

case 2: $G \geq \frac{B}{2}$

given that $OPT \leq B$,

$$\frac{OPT}{2} \leq \frac{B}{2} < G$$

□

1.6 Heuristic

1.7 Set cover

Given a universe U and a family of candidate sets $\mathcal{S}_U$, each $S \in \mathcal{S}_U$ has a non-negative cost $\text{cost}(S)$. The weighted set cover problem asks for a subfamily $S' \subseteq \mathcal{S}_U$ such that

$$\bigcup_{S \in S'} S = U,$$

while minimizing $\sum_{S \in S'} \text{cost}(S)$.

Greedy score

Let R be the set of elements that remain uncovered. For every set that can still cover at least one new element, define

$$\text{score}(S) = \frac{\text{cost}(S)}{|S \cap R|}.$$

This is the average cost per newly covered element. Greedy repeatedly chooses a set of minimum score.

Input: Universe U , candidate family $\mathcal{S}_U$ with costs

Output: A set cover $S' \subseteq \mathcal{S}_U$

$R \leftarrow U$ and $S' \leftarrow \emptyset$;

while $R \neq \emptyset$ **do**

$A_i \leftarrow \arg \min_{\substack{S \in \mathcal{S}_U \\ S \cap R \neq \emptyset}} \frac{\text{cost}(S)}{|S \cap R|};$
 $S' \leftarrow S' \cup \{A_i\};$
 $R \leftarrow R \setminus A_i;$

return S' ;

We prove

$$\text{ALG} \leq H_n \text{OPT} \leq (1 + \ln n) \text{OPT}, \quad H_n = \sum_{j=1}^n \frac{1}{j}, \quad n = |U|.$$

Proof. Assume the instance is feasible, so $\bigcup_{S \in \mathcal{S}_U} S = U$. The invariant is

$$U \setminus R = \bigcup_{A \in S'} A :$$

the elements removed from R are exactly those covered by the sets already selected. If $R \neq \emptyset$, feasibility guarantees a candidate set with $S \cap R \neq \emptyset$, so every iteration removes at least one element from R . Hence the algorithm terminates with $R = \emptyset$ and S' is a cover.

Blue: express the greedy cost exactly. **Red:** bound prices using OPT. **Purple:** combine the two.

1. Rewrite the greedy cost as element prices

Let R_i be the set of uncovered elements at the beginning of iteration i , and let A_i be the set selected in that iteration. The elements newly covered in this iteration are exactly $A_i \cap R_i$. For each $e \in A_i \cap R_i$, define

$$p(e) = \frac{\text{cost}(A_i)}{|A_i \cap R_i|}.$$

This is an accounting price, not an intrinsic cost of e . Already-covered elements of A_i are not charged again. The prices assigned in iteration i recover exactly the cost of A_i :

$$\sum_{e \in A_i \cap R_i} p(e) = |A_i \cap R_i| \frac{\text{cost}(A_i)}{|A_i \cap R_i|} = \text{cost}(A_i).$$

Every element is newly covered exactly once, so summing over all iterations gives the exact identity

$$\boxed{\text{ALG} = \sum_{A \in S'} \text{cost}(A) = \sum_{e \in U} p(e).}$$

2. Use one optimal set to bound its elements' prices

Let $\mathcal{O} \subseteq \mathcal{S}_U$ be a fixed optimal cover and define

$$\text{OPT} = \sum_{S \in \mathcal{O}} \text{cost}(S).$$

Fix any one set $S^* \in \mathcal{O}$ and let $m = |S^*|$. The set S^* may or may not also be selected by greedy; the proof only needs it to be an available candidate for comparison.

Order its elements by when greedy first covers them:

$$S^* = \{e_1, e_2, \dots, e_m\}.$$

If several elements are first covered in the same iteration, break the tie arbitrarily. Let $j(t)$ be the iteration in which e_t is first covered and define

$$r_t = |S^* \cap R_{j(t)}|.$$

Immediately before that iteration, at least $e_t, e_{t+1}, \dots, e_m$ are still uncovered. Therefore

$$r_t \geq m - t + 1.$$

At iteration $j(t)$, greedy chooses $A_{j(t)}$ with minimum score. Comparing its score with the available candidate S^* gives

$$\begin{aligned} p(e_t) &= \frac{\text{cost}(A_{j(t)})}{|A_{j(t)} \cap R_{j(t)}|} \\ &\leq \frac{\text{cost}(S^*)}{|S^* \cap R_{j(t)}|} = \frac{\text{cost}(S^*)}{r_t} \\ &\leq \frac{\text{cost}(S^*)}{m - t + 1}. \end{aligned}$$

The same inequality handles simultaneous coverage: tied elements are all still in $R_{j(t)}$ before that iteration, making r_t larger and the bound only stronger. Summing over the elements of this one optimal set,

$$\begin{aligned} \sum_{e \in S^*} p(e) &= \sum_{t=1}^m p(e_t) \\ &\leq \text{cost}(S^*) \sum_{t=1}^m \frac{1}{m - t + 1} \\ &= \text{cost}(S^*) H_m \leq \text{cost}(S^*) H_n. \end{aligned}$$

This is a bound on the *accounting prices of elements in S^** ; it is not a separate cost paid by greedy to cover only those elements.

3. Bridge the optimal-set bounds back to ALG

Every $e \in U$ belongs to at least one set in the optimal cover $\mathcal{O}$. Thus the middle sum below counts every $p(e)$ at least once (and may count it more than once when optimal sets overlap):

$$\begin{aligned} \text{ALG} &= \sum_{e \in U} p(e) \\ &\leq \sum_{S^* \in \mathcal{O}} \sum_{e \in S^*} p(e) \\ &\leq \sum_{S^* \in \mathcal{O}} H_n \text{cost}(S^*) \\ &= H_n \text{OPT}. \end{aligned}$$

Therefore greedy weighted set cover is an H_n -approximation, and since $H_n \leq 1 + \ln n$, it is also a $(1 + \ln n)$ -approximation. $\square$

1.8 Vertex Cover as a special case of Set cover

Vertex cover can be seen as a special case of set cover with additional structure. In set cover we need to cover all elements, while in vertex cover we are covering edges. The additional structure is the fact that each edge, the element we want to cover, only exist in exactly 2 "sets", which are the vertices.

Stronger structure may give better approximation

```

Input:  $G = (V, E)$ 
Output:  $n$ 
 $M \leftarrow \emptyset$  while  $E \neq \emptyset$  do
    Choose any remaining  $e \in E$ ;
     $M \leftarrow M \cup \{e\}$ ;
    foreach  $e' \in E, e' \text{ incident to } e$  do
         $E \leftarrow E \setminus \{e'\}$ ;
return  $M$ ;

```

We can show that this simple algorithm yields 2-approximation.

Proof. Feasibility

First we need to show the correctness (feasibility) of this algorithm. Consider otherwise, there exist uncovered edges, meaning both endpoints u, v of the edge do not incident to a matched edge $e \in M$, meaning $\{u, v\}$ is disjoint, and therefore could be added to M , contradicting to the maximality of M .

Guarantee

A Matching lower-bounds every possible vertex cover, because matching edges are disjoint, so a cover needs a distinct chosen endpoint for each one, i.e. at least one endpoint for each match.

$$|M| \leq \text{OPT}$$

Our algorithm takes both endpoint, thus

$$\text{ALG} = 2 \cdot |M| \leq 2 \cdot \text{OPT}$$

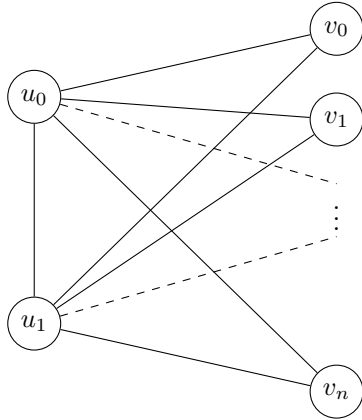
$\square$

Tutorial 1

Q1

Counter example of naive random start greedy algorithm

$$G = (V, E), \quad V = \{u_0, u_1\} \cup \{v_i \mid i \in [0, n]\}, \quad E = \{(u_0, u_1)\} \cup \{(u_0, v_i), (u_1, v_i) \mid i \in [0, n]\}$$



Worst case scenario (u_0, u_1) is never removed, resulting n edges removed instead of 1.

Simple greedy solution gives 3-approx

Similar to vertex cover, here we are looking for "triangle cover". Similar to a maximal matching set, we can find a maximal set of edge-disjoint triangles.

```

Input:  $G = (V, E)$ 
Output: Set  $R$  of edges to remove
 $R \leftarrow \emptyset$ ;
while  $G$  contains a triangle do
    Find any triangle  $(u, v, w)$  in  $G$ ;
     $R \leftarrow R \cup \{(u, v), (v, w), (u, w)\}$ ;
    Remove edges  $(u, v)$ ,  $(v, w)$ , and  $(u, w)$  from  $G$ ;
return  $R$ ;
```

Proof. Feasibility

Trivial

Guarantee

In the optimal solution, every disjoint triangle needs to have at least 1 edge removed in OPT , suppose there are k disjoint triangles,

$$k \leq OPT$$

Our algorithm remove all edges of all disjoint triangles,

$$ALG = 3k \leq 3 \cdot OPT$$

□

Patching with heuristic function of each edge

Rank all candidate edges by the number of triangles that can be removed. This can be done in polynomial time:

```

Input:  $G = (V, E)$ 
Output: Number of edges removed  $n$ 
 $H \leftarrow \emptyset$ ;
triangleCount  $\leftarrow \emptyset$ ;
foreach  $e = (u, v) \in E$  do
     $H[u] \leftarrow H[u].append(v)$ ;
     $H[v] \leftarrow H[v].append(u)$ ;
while any edge has triangle count  $> 0$ : access the edge with most triangles, remove, update,  $n++$  return  $n$ ;
```

1.9 Fully polynomial-time approximation scheme

Some pseudo polynomial dynamic programming algorithm can be converted to fully polynomial time approximation algorithm by sacrificing some precision to make pseudo polynomial term polynomial to the problem size.

1.9.1 Woeginger FPTAS

Given an algorithm in the form of

Input: vectors $x_1, x_2, \dots, x_n$, where each vector is made of non negative integers not bounded by n

- $\epsilon :=$ the required approximation ratio
- G : there exists an integer $G \geq 0$
- $1 + \frac{\epsilon}{2Gn}$

Example: Knapsack

First we need to derive given the pseudo polynomial dynamic programming solution that runs in $O(nW)$, we can reduce the number of columns in the DP table, by reducing the resolution. Given an $\epsilon > 0$, max price of any single item $v_{\max}$, define:

- $K = \frac{\epsilon v_{\max}}{n}$
- For each item x_i , define $v'_i = \lfloor \frac{v_i}{K} \rfloor$.

We are defining a rounding down function that convert unbounded variable v_i into v'_i that is bounded by an order of n :

$$f : v_i \mapsto \lfloor \frac{nv_i}{\epsilon v_{\max}} \rfloor$$

$$v'_i \leq \frac{n}{\epsilon}$$

The range of total value becomes

$$R : [0, nv_{\max}] \mapsto [0, n \times \lfloor \frac{n}{\epsilon} \rfloor]$$

Where ϵ is the parameterized approximation factor, and $v_{\max}$ is constant. We can transform the original bound $O(n^2W) = O(n^2v_{\max})$ to $O(n^2 \lfloor \frac{n}{\epsilon} \rfloor)$

First we make the claim that this approximation scheme is feasible (correctness proof)

$$ALG \geq (1 - \epsilon) \cdot OPT$$

Proof. Let $\mathcal{O}$ be the optimal choice set, for any item i , we are rounding down the value, meaning

$$\forall i \in \mathcal{O}, v_i \leq \frac{n}{\epsilon}$$

$$\sum_{i \in \mathcal{O}} v_i - K \times \sum_{i \in \mathcal{O}} v'_i \leq nK$$

□

1.10 Randomized approximation algorithm

1.11 Example: (Minimum) Vertex Cover

1.11.1 Greedy approach

Edge-based greedy selection yields us 2-approximation.

```

Input:  $G = (V, E)$ 
Output:  $V' \subseteq V$ 
 $V' \leftarrow \emptyset;$ 
foreach  $e = (u, v) \in E$  do
  if  $u \in V'$  or  $v \in V'$  then
     $\perp$  continue;
  else
     $b \leftarrow \text{Random}(0, 1);$ 
     $z \leftarrow \emptyset;$ 
    if  $b = 0$  then
       $\perp$   $z \leftarrow u;$ 
    else
       $\perp$   $z \leftarrow v;$ 
     $V' \leftarrow V' \cup \{z\};$ 
     $V \leftarrow V \setminus \{z\};$ 
    foreach  $e' = (u', v') \in E$  do
      if  $z = u'$  or  $z = v'$  then
         $\perp$   $E \leftarrow E \setminus \{e'\};$ 
   $\perp$ 
return  $V';$ 

```

Proof. Consider the optimal (minimum) vertex cover $V_{OPT} \subseteq V$, and the greedily chosen V_{ALG} . Since our algorithm guarantees that

$$\forall e = (u, v) \in E, u \in V_{ALG} \vee v \in V_{ALG}$$

The same applies to the optimal cover, i.e.

$$\forall e = (u, v) \in E, u \in V_{OPT} \vee v \in V_{OPT}$$

□

1.12 Example: MAX-SAT

Similarly to vertex cover, a simple random assignment gives us 2-approximation. Given a CNF formula

$$\varphi = \bigwedge_{i=1}^n \bigvee_{j \in [1, m]} l_j$$

with n clauses, and m boolean variables, a random assignment assigns each variable True or False.

```

Input: CNF  $\varphi$ 
Output: Assignment  $A \in \{0, 1\}^m$ 
 $A \leftarrow \emptyset;$ 
foreach  $v \in \varphi.V$  do
   $b \leftarrow \text{Random}(0, 1);$ 
   $A[v] = b;$ 
return  $A;$ 

```

Proof. Define independent probability variable p_i for each variable in φ

□

1.13 Inapproximability